Open Agent Hackathon: The Tech, Explained

How Zetaris, Meterless, Cursor and NVIDIA work, what they're good for, and ideas nobody else will pick

1. The level-2 idea: an "agent system" is just your code, with LLMs in the loop

You know what an LLM does. The leap to agents is realizing that **the "system" is ordinary code you write, and the LLM is called from inside it.** Three mechanics make that work.

1. Tool calling: how an LLM "does" things. An LLM can't run a database query itself. Instead, you tell it "you have a tool called `query_data(sql)`," and it replies with a structured request: *"call query_data with `SELECT ...`"*. **Your code** runs the query, hands the rows back, and the LLM continues. Every "the agent searched / looked up / checked" works this way. The LLM decides, and your code executes.

2. An "agent" is a role, not a product. In practice, an agent is a function in your code that calls an LLM with three things:

- **a role prompt**, e.g. "You are the Verifier. Reject any claim not backed by data";
- **a short list of tools** it's allowed to use;
- **a slice of the shared state** it needs to see.

Four agents can be four functions calling the same model with different instructions and tools.

3. Orchestration is plain `if` statements and loops. Your code decides who goes next based on what came back:

```
plan = planner(question)
for attempt in 1..2:                      # cap retries so it never loops forever
    evidence = retriever(plan)            # uses Zetaris
    verdict  = verifier(evidence)         # can reject
    log(verdict)                          # the trace judges will read
    if verdict.ok: break
    plan = planner.revise(plan, verdict.feedback)   # a loop BACK, not a straight line
answer = writer(evidence)
```

That loop **back** is the difference between a "pipeline" (judges mark it down) and an "agent system" (judges reward it). The extra ingredients the judges look for, all of them just code patterns:

- **Shared state:** one object, the "case file," that every agent reads and writes.
- **Authority:** the Verifier can block the Writer.
- **Branching:** high-risk results go to a different agent.
- **Fallback:** if a source is down, use sample data and *say so*.
- **Traces:** one log line per step, recording who acted, what they decided, who they handed to, how confident they were, and whether it succeeded or was rejected.

Where each sponsor plugs in:

Layer	What it does	Sponsor
The **builder**	Writes and edits the code with you	**Cursor**
The **orchestrator**	Your code: the loops, ifs and shared state	You (written in Cursor)
The **brains**	The LLM calls each agent makes	**NVIDIA Nemotron**, routed via **Meterless Runtime**
The **organs**	Memory, world state, safety checks	**Meterless Engines**
The **data door**	Every lookup of real data	**Zetaris**

2. Zetaris: one SQL front door to data that lives in many places

The problem it solves. Real data is scattered: a spreadsheet here, a database there, a feed from a website. The usual fix is to copy everything into one big database (a "warehouse"), which is slow, expensive, and always out of date. **Zetaris leaves data where it lives** and lets you query all of it as if it were one database.

How it works, step by step:

1. **Connect sources.** You register each source once: CSV/JSON files, Postgres, S3 storage, Snowflake, Salesforce, BigQuery, live REST feeds and more.
2. **It catalogs them.** Zetaris scans each source and lists its tables and columns in a **data catalog** (a searchable inventory of what data exists).
3. **Build a "virtual data mart" (optional).** This is a named bundle of tables from different sources, e.g. `care_team` = pharmacy file + clinic database + family notes. It's "virtual" because nothing is copied; it's a saved view.

4. **Query with plain SQL, across sources.** You can join a CSV to a database in one query. Behind the scenes, a **Query Director** splits the query and sends each piece to the right engine (Spark, Presto or Trino: heavy-duty query engines you never touch directly).
5. **Governance on top.** Roles and permissions control who can read what. Lineage records where each number came from.

How your agents talk to it (three doors):

- **REST API:** your code logs in and gets a token (a temporary pass). It sends `{"select": "SELECT ... ", "pageLimit": 100}`, gets back a query ID, pages through the rows, then closes the query. Simplest for a Retriever agent.
- **MCP:** MCP (Model Context Protocol) is a standard "plug" that lets an LLM use tools directly. Zetaris's MCP server exposes tools like `list_sources` and `query`, so the LLM itself can browse the catalog and write queries. You can also plug it into **Cursor**, so your coding AI can see your data while it builds.
- **JDBC/ODBC:** classic database connections, used by BI tools like Tableau.

Its newer "for AI" features: a **semantic layer** stores business definitions ("an *active patient* means a visit in the last 12 months"), so every agent uses the same meaning. Zetaris markets this as an "agentic lakehouse" (a lakehouse is a data warehouse built on top of cheap file storage).

Practical uses in an agent system:

Pattern	What the agent does with Zetaris	Why judges like it
Retriever	Turns the plan into SQL and fetches evidence	Basic, but required
Source discoverer	Calls `list_sources` and decides *which* sources matter for this question	Literally the "discover fragmented sources" track
Verifier	Re-runs *its own* query to check every number another agent cited	The docs' favourite "unique" move
Reconciler	Joins the same person or thing across sources and flags disagreements	The disagreement *is* the insight
Privacy guard	Permissions let an agent see only allowed columns (e.g. no names or dates of birth)	A big deal for health data, and a rare story

Load-bearing test: if you deleted Zetaris, would the agent still work? If yes, it's decoration. *Caveat:* Zetaris's public docs are thin on exact API details. Use the HackOS sponsor folder and the Data Day recording for setup.

3. Meterless: two different things under one name

A) Meterless Runtime: a switchboard for model calls

What it is. A program that runs on a computer and pretends to be OpenAI's API. Your code sends its LLM calls there, at `http://localhost:8088/v1`, and Meterless decides which actual model answers. That model can run on your own machine ("local") or in the cloud.

Why "OpenAI-compatible" matters. Almost every agent framework already knows how to talk to OpenAI. Pointing at Meterless instead means changing **one address and one key**, not rewriting code.

Routing modes instead of model names. You ask for a *kind* of thinking, and Meterless picks the model:

- `meterless/fast`: quick and cheap, for routing and formatting;
- `meterless/deep`: careful, for the Verifier;
- `meterless/local`: never leaves the machine;
- also `cheap`, `fusion` (combines models), and classes like `judge`, `vision` and `embed`.

Its catalog includes **NVIDIA Nemotron**, which is how NVIDIA slots in.

Practical use: give each agent the brain it needs, e.g. "Planner = fast, Verifier = deep." For health or elder-care data, **local mode is a privacy story**: sensitive notes never leave the device.

Mac caveat: the full Runtime app (Nightshift) runs on Windows. Mac support is limited for now (small models only). Ask the Meterless Help channel early how Mac users should connect.

B) Meterless Engines: blueprints for agent "organs"

The key concept. The engines are **not software you install.** Each one is a detailed **spec** (a precise written blueprint, in a file called `AGENTS.md`) plus a **conformance test** (an automatic checker). You hand the spec to Cursor, Cursor builds the engine *into your project*, and you're done when the test passes. It's like giving a contractor an architect's drawings plus an inspector's checklist. All four are written for **TypeScript** (a common web language), so plan to build in TypeScript.

Engine	What it is	How it works	Practical job in your build
H-MEM (memory)	Long-term memory	Every memory stores its **source** and a **confidence** score.	An agent that remembers

Engine	What it is	How it works	Practical job in your build
	that is *auditable*	Feedback adjusts confidence ("helpful" +, "wrong" −). Contradictions don't delete anything; the old memory is marked "superseded." A "sleep" step tidies memory but only *proposes* changes until a human approves. A trust ledger logs every read and write.	a person, a past decision or a preference, and can show *where it learned it*
World Model (shared state)	The shared case file, done properly	An append-only **event log** (nothing is overwritten, so you get full history). **Entities** (people, meds, appointments) get stable IDs so duplicates merge. A "context builder" assembles just what an agent needs within its token budget.	The single source of truth all your agents read and write. This is your "shared state" box, ticked properly.
Markovian (long runs)	Keeps long tasks from bloating	Splits work into chunks. Each chunk passes a short carryover note forward: *completed / remaining / decisions / context*.	For agents that loop many times, without slowing down or getting expensive
Scout Intent (safety and routing)	A bouncer before any action	Classifies what's being asked, scores the risk, and checks for prompt injection and personal data. It then issues a **signed "execution contract"** (a tamper-proof permission slip) naming the exact tool and inputs allowed. Low confidence means "ask a clarifying question" instead of acting.	A built-in risk agent with real authority to block. Ideal when actions could hurt someone.

How they connect: Scout approves the action, World Model holds the facts, H-MEM remembers across sessions, and Markovian keeps long runs lean.

The agentathon starter: a small TypeScript "research notebook" that already uses H-MEM (`remember`, `ask`, `audit`, `sleep --preview`). It does **not** include Zetaris or the Runtime, so you add those. It's a working base to grow from, not a finished multi-agent system.

Practical rule: routing calls through the Runtime is the minimum. **Using one or two engines for a real job, visible in your traces,** is what scores.

4. Cursor and NVIDIA

Cursor is a code editor (built on VS Code, the most popular one) with an AI coder inside. Its modes:

- **Ask:** questions only; it changes nothing. Use it for "explain this file."
- **Plan:** researches your project, asks clarifying questions, and writes a step-by-step plan you edit *before* any code is written.
- **Agent:** edits files and runs commands. You review the changes and accept or reject them.
- **Debug:** chases a bug by adding logging and testing guesses.

Cursor features that matter for you:

- **Rules:** a file of standing instructions Cursor follows every time, e.g. "TypeScript; every agent writes a trace line; never hardcode answers."
- **MCP:** connect Zetaris's MCP so Cursor can see your real tables while it writes queries.
- **Feeding it specs:** this is exactly how the Meterless engines are meant to be built. *"Implement `engines/hmem/AGENTS.md` until the conformance tests pass."*

How a 2.0-level builder should drive it:

1. Plan mode first.
2. One small task per request.
3. Ask it to explain before it changes anything.
4. Run the code after every step.
5. Commit (save a snapshot) whenever something works.

You must describe how you used Cursor in your submission, so jot notes as you go.

NVIDIA Nemotron is NVIDIA's family of open AI models, and you get free credits. You can call them through **Meterless** (simplest: just pick them in the routing) or directly from NVIDIA's API catalog at build.nvidia.com, which is also OpenAI-compatible. A natural split: **a big Nemotron for the Verifier or judge, a small one for the Planner.**

5. Ideas a room full of engineers won't think of

These domains are an edge: real, emotional problems with obvious users and real stakes. Always use **fake data** for people, and frame outputs as **preparation for a clinician or caregiver**, never diagnosis. Judges reward that honesty. *Tracks: 1 = Fragmented Intelligence, 2 = Explain Why, 3 = Reasoning Architecture.*

Older adults and memory

Idea	Track	How the tech does real work
Caregiver Handoff Brief. Three siblings share care of Mom; each has a different piece of the picture.	1	**Zetaris** joins the pharmacy list, clinic visit notes, a shared family log and the calendar. A **Reconciler** finds conflicts (two medication lists disagree, which is a real, dangerous problem). A **Verifier** re-queries before anything is stated, and conflicts escalate to a human. Output: today's one-page brief, with every fact sourced.
Memory Companion for early Alzheimer's.	3	**H-MEM** holds the person's life facts, with source and confidence ("daughter Sarah said, 0.9"). **World Model** tracks meds and appointments. When she asks "did I take my pill?", the agent checks the dose log in **Zetaris**, never a guess. **Scout Intent** blocks risky actions ("send money to my grandson") and escalates to family.
Is This a Scam? for older adults.	2	A suspicious message is checked against public scam-pattern lists and the person's own contacts and accounts (via **Zetaris**). Agents argue *for* and *against* "scam," and the output explains *why*, in plain words.

Fertility, pregnancy and parenthood

Idea	Track	How the tech does real work
IVF Clinic Reality Check	1 or 2	**Zetaris** joins the CDC's public clinic-by-clinic IVF success data, clinic services and profiles, a state insurance-mandate table and cost data. A **Critic** enforces the CDC's own warning that raw clinic comparisons mislead, and adjusts for the patient's age. Output: a shortlist with reasons and caveats.
Meds While Pregnant or Breastfeeding: Evidence Brief	2	The NIH's **LactMed** database (free bulk download) and **openFDA** drug labels (free API) are joined in **Zetaris**. A **Verifier** refuses any claim without a citation, and **Scout Intent** flags high-risk drugs as "talk to your clinician." Output: a one-page brief to bring to an appointment.
Parental Leave Maximizer	3	State paid-leave rules, FMLA (the federal job-protected leave law), the employer handbook and the due date → a leave plan with deadlines and paperwork. A **Critic** catches rules that conflict, and **H-MEM** remembers what HR already approved. Low medical risk and a huge real pain point.
Fertility Insurance Denial Decoder	2	The denial letter, the plan document and the state mandate → *why* it was denied, which clause applies, and a drafted appeal. The Verifier checks every clause it cites.

My picks for you: the **Caregiver Handoff Brief**, which is the cleanest fit for Zetaris, a natural critic loop, and an obvious user. Or the **Memory Companion**, which is the best showcase for the Meterless engines, since a memory product built on a memory engine is an easy story to tell.

To sharpen any idea, fill in: *helps [specific person] with [specific moment] by [what the agents check and decide] using [2–4 sources], producing [one thing they'd actually use].* Then ask:

- Where's the loop **back**?
- Who has authority to say **no**?
- What would a **wrong answer cost**?

Sources: the 10 official hackathon docs in this folder; github.com/Meterless/Meterless (AGENTS.md, engine specs, agentathon-starter), meterless.ai; zetaris.com and kbase.zetaris.com; Cursor docs and 2026 guides; CDC ART datasets (catalog.data.gov); LactMed (NCBI); openFDA. Vendor details may change, so confirm in the HackOS sponsor folders.